Volentify

System reference: what is built, how it works, and where every piece of data comes from

Component	State on 2 November 2026	Detail
Website	**LIVE**	https://volentify.vercel.app · Vercel Web Analytics enabled
API	**LIVE**	FastAPI on Vercel · /api/health reports ONLINE
Database	**ONLINE**	Neon PostgreSQL (read from DB_POSTGRES_URL) · Supabase fully removed
Live hazard feeds	**LIVE**	GDACS, USGS, NASA EONET every 15 min · 10 active incidents at the time of writing
Sample data	**FIXED**	None shipped; the old demonstration rows were purged from Neon by migration
Android app	**LIVE**	Volentify Mesh mesh-v1.0.6 (built and unit-tested by CI) · download at /mesh
Cyclone models	**MEASURED**	volentify-cyclone-ni@2026.11.02 · trained 2026-11-02
Dispatch benchmark	**MEASURED**	generated 2026-11-02 · reruns weekly
Research archive	**LIVE**	180 records so far · collected every 3 h
Google sign-in · Gemini	**LIVE**	client configured · gemini-3.5-flash briefings

Repository: github.com/Gyanvir/Volentify (public) · version 3.5.0 · commit 35f124e. Figures are read from the repository's model card and benchmark files and from the live site at the time of writing. Where something is a target rather than a measurement, it says so. This edition supersedes the 1 November 2026 reference.

Contents

1. What Volentify is
2. Architecture
3. Technology used
4. Where the code lives
5. The website, page by page
6. The API, endpoint by endpoint
7. Where ML and AI are, and where they are not
8. The cyclone ML pipeline
9. Dispatch engine and its benchmark
10. Where every piece of data comes from
11. Storage, migrations and the research archive
12. Volentify Mesh: the Android relay app
13. Security, privacy, testing and operations
14. What changed in this edition
15. Where we are

1. What Volentify is

Volentify is a disaster-intelligence and volunteer-coordination platform for India. It gives incident commanders, NGOs, volunteers and citizens one shared picture of an emergency, built only from real, current data, and keeps field reporting working when mobile networks fail.

It is built around four loops, and every feature belongs to one of them:

Loop	What happens	Built with
See	Live incidents from public hazard feeds, hazard zones, satellite imagery on the tactical map, radar, official warnings in force, hospitals and shelters near incidents.	Tactical map (MapLibre), live feeds (GDACS, USGS, NASA EONET), NDMA SACHET, OpenStreetMap
Report	Citizens report from the web; volunteers report from phones, even with no network, through Bluetooth relaying. Phones upload automatically whenever they get mobile data or Wi-Fi.	Report form, Volentify Mesh Android app, mesh ingest API
Decide	Reports are linked to the incident they fall inside, triaged by language and urgency, duplicates merged, briefings written, cyclones forecast.	Incident linking, Laya triage, dedup, briefing, cyclone ML models, storm-surge estimator
Act	Missions are created and the right responders dispatched; specialists are protected from routine work.	Sperling dispatch engine, command hub, volunteer dispatch board, alerts

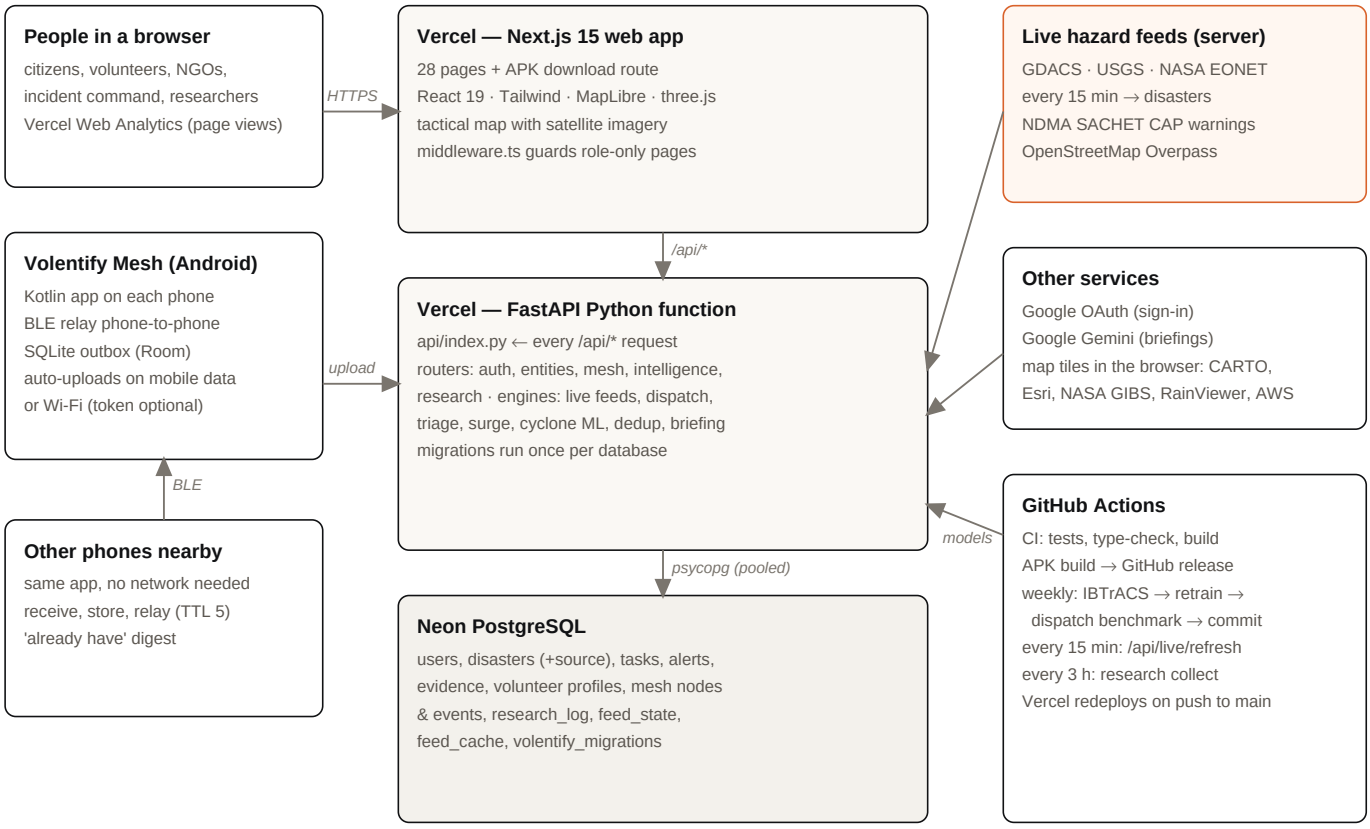
Who uses it

Role	What they can do	Verification
Citizen	See incidents, alerts and resources; submit reports; hold a civilian SOS pass.	None
Volunteer	Everything a citizen can, plus accept missions, set availability, get an optional gateway token for the mesh app, hold a responder muster pass.	None
NGO member	Command hub: review mesh reports, create missions, run dispatch, broadcast advisories.	Admin verifies
Incident admin	Everything, plus the admin console (users, roles, verification, system health) and research export.	Admin verifies (or listed in VOLENTIFY_ADMIN_EMAILS)
Researcher	Read-only analysis plus research archive export.	None

Every account completes a five-step onboarding (mobile number, role, role details, safety profile, review) and receives a signed digital muster pass.

2. Architecture

One Vercel project serves both the website and the API from the same domain, so there is no CORS, one deployment and one set of environment variables. The Android app is the only other runtime.



How a request flows

- A browser loads a Next.js page from Vercel. Pages fetch data from /api/… on the same domain and poll it (mesh reports every 10 s; unchanged responses do not redraw).
- vercel.json rewrites every /api/* request to one Python function, api/index.py (FastAPI). It answers from PostgreSQL, calls the engines, or fetches an outside source.
- Incident lists refresh the live hazard feeds when they are more than 15 minutes old. A feed_state row in the database ensures only one serverless instance refreshes at a time; a GitHub workflow calls /api/live/refresh every 15 minutes so visitors rarely wait.
- If the database is unreachable, reads fall back to an in-memory mirror of the live feeds (never invented data) and every response carries an X-Data-Source header (database / fallback / memory) so screens can say "Database offline" or "Not saved". Writes that must not be lost (mesh ingest) return 503 instead, so phones keep their outbox and retry.
- On first connection the API creates the schema and applies pending migrations once, under an advisory lock so parallel serverless instances cannot race.
- Phones relay reports to each other over Bluetooth; whichever phone gets mobile data or Wi-Fi uploads its whole outbox to /api/mesh/ingest, with or without a gateway token.
- GitHub Actions does all scheduled work: feed refresh, retraining, benchmarking, building the APK and collecting the research archive. Vercel redeploys on every push to main.

3. Technology used

Layer	Technology	Version	Used for
Web framework	Next.js (App Router)	15.5	Pages, routing, server rendering, middleware
UI library · language	React · TypeScript	19.1 · 5.7	All interface components, type-checked
Styling · type	Tailwind CSS tokens · Instrument Serif, Inter, IBM Plex Mono	3.4	Design system (docs/DESIGN.md)
Maps	MapLibre GL JS	4.7	2D/3D tactical map, satellite base maps, terrain, measuring, export
3D · charts · icons	three.js · Recharts · lucide-react · qrcode	0.171 · 3.2 · 0.469 · 1.5	Globe and HUD · analytics · icons · APK QR code
Analytics	Vercel Web Analytics (@vercel/analytics)	2.0	Privacy-friendly page-view counts
Route protection	jose (JWT verify at the edge)	5.9	middleware.ts role checks
API	FastAPI + Pydantic on Vercel Python	≥0.115 · ≥2.10 · 3.12	All /api endpoints and request validation
Database driver	psycopg 3 (binary)	≥3.2	Neon pooled endpoint (PgBouncer, transaction mode)
Auth	PyJWT[crypto] · stdlib scrypt	≥2.9	Sessions, Google ID-token checks, passwords
HTTP client	httpx	≥0.27	GDACS, USGS, EONET, SACHET, Overpass, Gemini
Database	PostgreSQL on Neon (via Vercel Storage)	15+	All durable data and the research archive
Schema	schema.sql + migrations.py (auto-applied) · Prisma schema	Prisma 6.19	Same schema two ways; Prisma optional
ML training	XGBoost · scikit-learn · pandas · NumPy · SciPy	3.4.1 · …	Cyclone models, cross-validation, exact dispatch baselines
ML serving	treeinfer.py (pure Python)	—	Runs XGBoost trees without installing XGBoost
LLM	Google Gemini 3.5 Flash (auto-updating)	AI Studio or Vertex key	Briefing text and report merging
Android	Kotlin · Jetpack Compose · Room · WorkManager · OkHttp · BLE GATT	2.0 · BOM 2024.09 · 2.6 · 2.9 · 4.12	Volentify Mesh app
Hosting · automation	Vercel · GitHub Actions	—	Web + API · CI, APK releases, training, feeds, archive
Tests	pytest · JUnit	—	API, ML pipeline and Android unit tests (section 13)

4. Where the code lives

```
Volentify/
■■■ README.md                  setup, deployment, live data, research archive
■■■ docs/PRD.md · DESIGN.md    product requirements v3.5 · design system
■■■ docs/build_system_reference.py   builds this PDF from the model files
■■■ .github/workflows/
■   ■■■ ci.yml                 tests + type-check + production build on every push
■   ■■■ android.yml            builds, tests and releases the Mesh APK
■   ■■■ ml-train.yml           weekly: IBTrACS, retrain, benchmark dispatch, commit
■   ■■■ live-feeds.yml         every 15 min: GET /api/live/refresh
■   ■■■ research-collect.yml   every 3 h: POST /api/research/collect
■■■ ml/volentify_ml/           data.py · features.py · train.py · dispatch_bench.py
■■■ mesh-android/              Volentify Mesh: data, mesh (BLE, digest, framing), sync, location, ui
■■■ frontend/                  the deployable app (Vercel root directory)
    ■■■ app/                   Next.js pages grouped by purpose; map; mesh; download
    ■■■ components/            ui, layout, map (TacticalMap, DetailDrawer), operations, globe, auth
    ■■■ lib/                   API client, session, types, constants, release lookup
    ■■■ middleware.ts          role-based page protection
    ■■■ prisma/schema.prisma   database schema (Prisma form)
    ■■■ api/index.py           FastAPI entry point and /api/health
    ■■■ api/_lib/core/         config, database (Neon + in-memory), security, research archive
    ■■■ api/_lib/engines/      live (feeds), intel (SACHET, OSM, imagery), dispatch, triage, surge, cy
clone …
    ■■■ api/_lib/routers/      auth, entities, mesh, intelligence, research
    ■■■ api/_lib/data/         schema.sql, migrations.py, seed (setup script), reference (helplines)
    ■■■ api/_lib/models/       trained models, model card, dispatch benchmark (written by CI)
    ■■■ tests/                 test_api.py and fixtures.py (fictional test-only dataset)
```

5. The website, page by page

Access is enforced twice: by middleware.ts before the page loads, and again by the API for every action.

Route	Page	What it does	Access
/	Home	Live incident count from the hazard feeds, God's Eye globe, entry points, app download.	Anyone
/map	Tactical map	Full-screen 2D/3D map: live incidents, hazard zones, mesh reports, missions, responders, nodes, hospitals, shelters; Satellite switch and base-map switcher (Esri, MODIS, VIIRS SNPP, VIIRS NOAA-20) with opacity, rain radar, terrain, filters, measuring, PNG export. /map?imagery=…&radar;=1 opens a preset.	Anyone
/disasters	Active incidents	Incidents from GDACS, USGS and NASA EONET, ordered by severity, with their source.	Anyone
/disasters/[id]	Incident dossier	Evidence timeline (source accounts and mesh reports), source link, missions, briefing, forecast.	Anyone
/alerts	Alerts	Volentify advisories, plus official CAP warnings in force from NDMA SACHET (expired ones removed).	Anyone
/volunteer	Volunteer dispatch	Open missions, accept, set availability, run and explain a dispatch cycle.	Anyone (actions need sign-in)
/resources	Emergency resources	National helplines; hospitals, shelters and assembly points mapped in OpenStreetMap near active incidents.	Anyone
/predictions	Forecasts	Storm-surge sandbox, cyclone model performance, dispatch benchmark, incident forecasts, triage playground.	Anyone
/disaster-intelligence	Satellite & bulletins	Imagery list that drives the live tactical map (operational layers stay on top), official bulletins, report corroboration on real warnings.	Anyone
/analytics	Analytics	Charts computed live from incidents, missions, responders and mesh reports.	Anyone
/mesh · /download/android	Mesh app	Download button, QR code, install steps; streams the latest APK release with the Android MIME type.	Anyone
/login · /register · /auth/callback	Accounts	Email + password or Google (nonce + state checked).	Anyone
/onboarding · /dashboard	Muster pass · desk	Five-step wizard and signed pass; role desk with missions, alerts and optional gateway token.	Signed in
/agency	Command hub	Mesh review queue (10 s refresh), mission form, dispatch run, advisory broadcast.	Verified NGO / admin
/admin	Administration	Users, roles, verification, database and system status.	Verified admin
/contact · /research · /about · /faqs · /news · /knowledge · /government · /ngo-partners · /privacy · /terms	Information	Report form, methods, questions, situation reports, preparedness, partners (organisations verified on the platform), legal.	Anyone

6. The API, endpoint by endpoint

46 endpoints under /api on the same domain; interactive documentation at /api/docs. Requests are validated by Pydantic; errors come back as one readable sentence.

Method · path	Purpose	Auth
GET /api/health	Status, database state (with a sanitised reason when DEGRADED), last live-feed refresh, configuration in use; ?deep=1 tests Gemini.	—
GET /api/live/status · /api/live/refresh	NEW. When each hazard feed last answered; refresh feeds older than 2 min and warm the facility cache (called every 15 min).	— (30/hour)
POST /api/auth/register · /login · /logout · Google start/callback	Email accounts (scrypt) and Google OpenID Connect; session cookie.	—
GET /api/auth/me · POST /api/auth/onboard	Current user; complete onboarding and issue the muster pass.	Session
GET · PATCH /api/admin/users	List users; change role or verification.	Admin
GET /api/disasters (?include_closed) · /disasters/{id} · /briefing	Live incidents; dossier with evidence, missions and alerts; situation briefing.	—
/api/tasks · /tasks/{id}/accept · PATCH /tasks/{id}	Missions: list, create, accept, update.	Varies
GET · POST /api/alerts	Volentify advisories: list; broadcast (command roles).	Varies
/api/volunteers · /volunteers/me · /volunteer/availability	Responders and availability.	Varies
GET /api/organizations · /api/resources	Organisations verified on the platform; helplines plus OSM hospitals and shelters near active incidents (cached 6 h).	—
POST /api/reports	Citizen report: triaged and linked to the nearest active incident.	— (6/min)
POST /api/predict · GET /api/ml/models	Storm surge + hazard probability with the trained cyclone models; model card and benchmark.	—
POST /api/triage/laya · /events/ingest · /briefing/generate	Triage; de-duplicate raw reports; corroborated briefing.	—
POST /api/match · /match/dispatch-batch	Rank responders for one mission; run a full dispatch cycle.	—
POST /api/infrastructure/nearby	Facilities near a point (Overpass); empty with mode OFFLINE when Overpass is unreachable.	—
GET /api/bhuvan/layers · /bhuvan/bulletins (?limit)	Imagery catalogue; CAP warnings in force (LIVE, CACHED from archive, or OFFLINE).	—
POST /api/mesh/token	Optional short-lived gateway token for a phone (240 min).	Volunteer / NGO / admin
POST /api/mesh/ingest	Phones upload reports and messages (≤100 per batch, ≤4 KB each, idempotent). Each report is linked to the active incident it falls inside and added to its evidence timeline.	Token optional (20/min without, 60/min with)
GET /api/mesh/events · /nodes · /incidents	Field reports (redacted for unverified readers), phones seen, incident areas to cache offline.	—
PATCH /api/mesh/events/{id}	Mark a report reviewed, or promote it to an official mission.	NGO / admin
POST /api/research/collect · GET /summary · /export	Archive CAP alerts, feed refresh, model version, platform snapshot; summary; CSV/JSONL export.	— (4/hour) · admin / researcher

7. Where ML and AI are, and where they are not

Only two components are trained machine-learning models. Everything else is an analytical model, an optimisation heuristic or a rule system, sometimes with an optional language model on top.

Component	Kind	Learns from data?	Where it runs	State
Cyclone 24-hour intensity change	Trained ML (XGBoost regression)	Yes: IBTrACS, 1980 to now	API, every forecast	MEASURED
Severe-storm probability (≥48 kt in 24 h)	Trained ML (XGBoost classifier)	Yes: IBTrACS	API, every forecast	MEASURED
Storm-surge height	Physical estimator	No (calibrated formula)	API, /api/predict	TARGET
Hazard probability	Logistic combination, fixed weights	No; combines the ML probability with rain and surge	API, /api/predict	PARTIAL
Live incident severity	Published scales (GDACS alert level, IMD cyclone classes, USGS magnitude/PAGER)	No	API, every feed refresh	LIVE
Sperling dispatch	Optimisation heuristic	No (scoring rules)	API, every dispatch	MEASURED
Laya triage, tier 3	Multilingual lexicon + rules	No	API, every report	LIVE
Laya triage, tiers 1–2	Local router package / hosted model	External	Only if installed / key set	OPTIONAL
Report de-duplication · incident linking	Spatio-temporal clustering · nearest incident within 1.5× radius	No	API	LIVE
Fact merging & briefing text	LLM (Gemini 3.5 Flash)	Pre-trained by Google	API, on request	LIVE
SitRep DistilBERT model	Planned trained ML	—	—	NOT BUILT

How the trained models reach users

- The weekly workflow trains with XGBoost and exports each model as a JSON tree dump; treeinfer.py evaluates them in pure Python so the Vercel function stays small. A parity test checks it matches XGBoost (inputs and thresholds rounded to 32-bit floats).
- /api/predict converts the request to the 14 features, runs both models, feeds the forecast peak wind into the surge estimator and the severe-storm probability into the hazard probability, and names the model, e.g. volentify-cyclone-ni@2026.11.02 + surge-analytic-v1.
- If the model files are missing, the API falls back to the analytical surge model alone and says so in the model name.

8. The cyclone ML pipeline

Step	What happens
1 · Download	NOAA NCEI IBTrACS v04r01, North Indian Ocean CSV, fetched fresh each week by ml/volentify_ml/data.py; the raw file is not kept.
2 · Clean	Wind from RSMC New Delhi (IMD) where reported, otherwise WMO, otherwise US agencies; six-hourly fixes; storms since 1980. This build: 463 storms, 8,559 fixes, 1980-10-10 to 2026-10-29.
3 · Features	14 features per fix (wind, pressure, position, distance to land, speed, heading sin/cos, season sin/cos, 6/12/24-hour wind change, Arabian Sea flag). Their order is a contract with the API.
4 · Targets	intensity_24h: change in max sustained wind (kt) over the next 24 h. severe_24h: storm reaches ≥48 kt (IMD severe cyclonic storm) within 24 h.
5 · Augment	History masking: each fix is also trained with its history hidden, so the model works from a single report.
6 · Evaluate	5-fold storm-grouped cross-validation (GroupKFold): every storm held out whole. Compared with persistence and climatology.
7 · Export · ship	Final models trained on all storms, written with model_card.json; tests run; the bot commits; Vercel redeploys. Every Sunday night IST and on pipeline changes.

Held-out results (this build)

Model	Metric	Model	Baseline	What it means
Intensity, 24 h	Mean absolute error	7.64 kt	9.67 kt (persistence)	21% lower error than assuming no change
Intensity, 24 h	MAE from one report only	8.02 kt	—	still works with no history
Severe within 24 h	ROC AUC	0.9604	0.5 (chance)	ranks storms that will turn severe very well
Severe within 24 h	AUC, storms still below 48 kt	0.8994	0.5	the hard, useful case: not yet severe
Severe within 24 h	Brier score	0.0572	0.1748 (climatology)	Brier skill 0.6727; base rate 23%
Severe within 24 h	Accuracy at 0.5	92.1%	77.4% (always "no")	accuracy alone flatters; prefer AUC and Brier

Samples: 6,819 intensity cases and 8,094 severe-storm cases. Trained with XGBoost 3.4.1.

9. Dispatch engine and its benchmark

The Sperling engine assigns open missions to available responders in strict priority order (critical, then high, moderate, low). Within a tier, each slot goes to the eligible responder with the best score:

$C(i,j) = W_{urgency} \cdot S_k \cdot Sim \cdot \Pi_{squander} \times 1/(1 + d/d0) \times 1/(1 + \beta \cdot H_{work})$

Term	Value in code	Meaning
$W_{urgency}$	10 / 5 / 2 / 1	critical / high / moderate / low
S_k	$1/\sqrt{N_k}$	scarcity: the fewer responders hold skill k, the higher the weight
Sim	1.00 / 0.55 / 0.10	exact / adjacent skill / generic
$\Pi_{squander}$	0.12	a rare specialist (skill held by ≤3 responders) sent to generic work
d0 · β	15.0 km · 0.15	distance at which the score halves · fatigue per active mission

Benchmark (2026-11-02, reruns weekly)

ml/volentify_ml/dispatch_bench.py calls the production engine unchanged and compares it with a nearest-responder baseline, the Hungarian optimum (ignores urgency order) and an exact priority MILP (HiGHS, 10 s limit), 5 scenarios per size.

Scale	Method	Time / cycle	Critical filled	Weighted fill	Score vs optimum	Specialists on generic	Timed out
40 responders · 46 slots	Volentify (Sperling)	2.9 ms	88%	83%	92%	14%	0%
	Nearest responder	0.9 ms	52%	69%	58%	34%	0%
	Exact optimum	6.0 ms	86%	83%	100%	14%	0%
	Exact priority MILP	26.9 ms	88%	83%	99%	14%	0%
150 responders · 169 slots	Volentify (Sperling)	33.6 ms	100%	90%	92%	0%	0%
	Nearest responder	10.8 ms	64%	75%	56%	13%	0%
	Exact optimum	74.7 ms	100%	91%	100%	0%	0%
	Exact priority MILP	496 ms	100%	91%	99%	0%	0%
500 responders · 544 slots	Volentify (Sperling)	364 ms	100%	94%	91%	0%	0%
	Nearest responder	115 ms	69%	77%	56%	0%	0%
	Exact optimum	808 ms	100%	94%	100%	0%	0%
	Exact priority MILP	11.3 s	—	—	—	—	100%
1000 responders · 1125 slots	Volentify (Sperling)	1.5 s	100%	91%	90%	0%	0%
	Nearest responder	453 ms	67%	75%	55%	0%	0%
	Exact optimum	3.3 s	100%	91%	100%	0%	0%
	Exact priority MILP	30.2 s	—	—	—	—	100%

- Sperling fills critical missions as well as the exact priority MILP wherever the MILP finishes, and far better than the nearest-responder baseline.
- Exact priority optimisation does not scale: from 500 responders the MILP hit its time limit in every run, while Sperling took 1.5 s at 1000 responders.
- Sperling gives up some total match score against the Hungarian optimum because it deliberately serves urgent missions first.
- Supersedes the PRD figures (0.32 ms, 64.2% MILP timeouts, 100% efficiency), which no benchmark in the code produced.

10. Where every piece of data comes from

Nothing shown on the platform is invented. "Live" means fetched from the source; "archived" means also written to the research archive.

Source	What we take	How and when	Stored?
GDACS (EC Joint Research Centre / UN OCHA)	Tropical cyclones, floods, droughts, wildfires, volcanoes in and around India (lat 4–38, lng 65–100): name, alert level, wind, countries, report link	Server, one request per hazard type, every ≤15 min	disasters (source=GDACS) + evidence
USGS Earthquake Hazards Program	Earthquakes M4.5+ in the last 7 days in the region: magnitude, depth, place, PAGER alert, tsunami flag	Server, every ≤15 min	disasters (source=USGS) + evidence
NASA EONET	Open natural events (severe storms with wind, wildfires, floods, landslides…); duplicates of GDACS cyclones merged	Server, every ≤15 min	disasters (source=NASA EONET) + evidence
NDMA SACHET	Official CAP warnings: English headline, CAP severity, event, area, instruction, expiry (each CAP message read)	Server, cached 5 min; expired warnings removed	Yes: research_log (cap_alert), each alert once
OpenStreetMap (Overpass + mirrors)	Hospitals, shelters, assembly points near the six most severe active land incidents; facilities near any point	Server, cached 6 h in feed_cache	feed_cache
NOAA NCEI IBTrACS v04r01	North Indian Ocean cyclone best tracks since 1980	Weekly training workflow	Trained models + model card (git)
Esri · NASA GIBS · RainViewer · CARTO · AWS Terrain	Satellite base maps (static mosaic, MODIS/VIIRS daily), radar, dark base map, elevation	Tiles loaded by the browser	No
Google · Gemini · Impossibl (optional)	Sign-in identity · generated briefing text · hosted triage	On sign-in · on request · if key set	users · no · no
Citizens (web)	Incident reports with location	Report form → /api/reports	evidence
Volunteers' phones	Mesh reports and messages, phone battery and position	Bluetooth relay → automatic upload on mobile data or Wi-Fi	mesh_events, mesh_nodes, evidence (linked incident)
Responders and organisations	Profiles, skills, availability, missions	Onboarding, dashboards, command hub	users, volunteer_profiles, tasks, organizations
Helplines	National emergency numbers (112, 108, 101, 100, 1078, 1070, 1077, NDRF, 1091, 1098)	Static reference (data/reference.py)	Code

Severity of live incidents follows published scales: GDACS alert level (Red → critical, Orange → high); IMD cyclone classes by sustained wind (≥118 km/h critical, ≥89 high, ≥62 moderate); USGS PAGER alert or magnitude (≥7 critical, ≥6 high, ≥5 moderate). Incidents a source stops listing are closed automatically; only incidents with recent activity are kept.

11. Storage, migrations and the research archive

Where	What is kept	How long
Neon PostgreSQL	organizations, users, volunteer_profiles, disasters (with source, source_url, external_id), tasks, evidence, alerts, mesh_nodes, mesh_events, research_log, feed_state, feed_cache, volentify_migrations	Until deleted; live incidents closed when dropped by their source
Git (GitHub)	Code, trained models, model card, dispatch benchmark; every weekly retrain is a commit	Permanent
GitHub releases	Every Mesh APK build (latest mesh-v1.0.6)	Permanent
Each phone (SQLite via Room)	Reports and messages created or relayed; settings; cached incident areas; confirmed reports kept 72 h	Until uninstalled
Browser	Session cookie (HTTP-only, 72 h); OAuth nonce/state cookies (10 min)	Short-lived

Migrations (api/_lib/data/migrations.py)

Migration	What it does
2026-11-live-feeds	Adds source, source_url and external_id to disasters, plus the feed_state and feed_cache tables.
2026-11-purge-demo	Removes the demonstration rows the earlier seed inserted (Cyclone "Remal", Brahmaputra, Wayanad, Chamoli and Delhi incidents, nine sample volunteers, sample missions, alerts, mesh nodes and reports, five organisation records). Real users' rows are kept; alerts and mesh reports that pointed at demo incidents are unlinked.

Each runs once per database, inside a transaction with an advisory lock, and is recorded in volentify_migrations.

Research archive (research_log)

Live observations and model outputs are archived with timestamps so results can be reproduced. Writes are best-effort and never slow the request that triggered them. No personal data is archived.

Kind	What is recorded	When	Records so far
cap_alert	Each NDMA SACHET warning, once	every feed pull and every 3 h	134
forecast	Inputs and output of every /api/predict call, with model version	every forecast	21
dispatch_run	Tier counts, completeness, latency, scarcity weights, score distribution	every dispatch cycle	18
model_version	Model card of each trained version	once per version	3
platform_snapshot	Active incidents, task and responder counts, mesh nodes, hop counts	every 3 h	4

Snapshots taken before 2 November 2026 counted the demonstration incidents; filter by recorded_at when analysing.

12. Volentify Mesh: the Android relay app

The core offline feature. Phones pass reports and messages to each other over Bluetooth Low Energy with no mobile network or Wi-Fi, and any phone that gets mobile data or Wi-Fi uploads everything it holds to the website, where it becomes incident data.

Aspect	How it works
Discovery	Every phone advertises and scans (low-latency mode) for the Volentify BLE service 7f3c0001-6a1e-4c2b-9b8e-5d1f0c7a9e10 and is both GATT server and client. Scans restart every 25 min (Android downgrades long scans).
Relay timing	Event-driven: a new or newly received report is offered at once to every phone seen in the last minute. No per-peer cool-down; a failed peer backs off 3 s doubling to 30 s. Two peers served in parallel, never two sessions to one phone.
'Already have' digest	Characteristic 7f3c0003-…: 4-byte prefixes of the ~120 newest report ids a phone holds. Peers read it first and send only what is missing (older app versions get everything).
Transfer	Inbox characteristic 7f3c0002-…; frames with a 6-byte header, capped at 512 bytes per write (ATT maximum); high connection priority and 2M PHY when supported; one retry on GATT error 133; up to 50 reports per session, 15 s session limit.
Envelope · relaying	event id, origin node, type, priority, payload ≤4 KB, location, observed time, hop count, TTL (max 5). Duplicates dropped by id; each relay stores hop+1, TTL–1; never echoed to the sender.
Upload	Automatic whenever the phone has validated internet (mobile data or Wi-Fi): an app-level network watcher and the relay service upload at once; WorkManager with a network constraint covers the app being closed. Batches of 100 to /api/mesh/ingest; a gateway token is optional, an expired one is dropped and uploads continue.
On the website	Each uploaded report is linked to the active incident it falls inside (within 1.5× its radius), appears on the tactical map and the incident's evidence timeline. Reports about people show only type and a 1–5 / 6+ headcount publicly.
Location	Fine and approximate location requested together (Android 12+ ignores fine alone). Fix taken from fused/GPS/network providers in parallel with accuracy; the report screen shows live status (±m, permission needed, location off, retry). Messages attach a best-effort fix (≤5 s). Optional ±500 m masking.
Screens	Home (status, Sync now), Report (with location row), Messages, Settings (name, server, optional token, mask location).
Build	Kotlin 2.0, Compose, minSdk 26, targetSdk 34. GitHub Actions builds, unit-tests and publishes each release; latest mesh-v1.0.6. Install from /mesh.

13. Security, privacy, testing and operations

- Sessions are HS256 JWTs in an HTTP-only, SameSite=Lax cookie (Secure in production), valid 72 hours; key from NEXTAUTH_SECRET or derived from the database URL.
- Passwords hashed with scrypt; Google sign-in verifies signature, audience, issuer and a one-time nonce.
- Elevated pages protected in middleware.ts and every action re-checked in the API; NGO and admin roles need verification.
- Rate limits per instance: login 10/min, citizen reports 6/min, mesh ingest 60/min with a token and 20/min without, live refresh 30/hour, research collection 4/hour.
- Field reports about people are redacted for anyone who is not a verified NGO member or admin, and never put free text into public incident timelines.
- /api/health shows why the database is degraded with host names and credentials stripped. Secrets live only in Vercel environment variables and the local .env (git-ignored).

Tests

Suite	Covers	Runs
frontend/tests (45 tests)	API contracts, auth, roles, mesh ingest (token-less uploads, incident linking, redaction), live-feed parsers and sync/close, CAP parsing, no-sample-data, degraded health, research archive	CI on every push; weekly after retraining
ml/tests (6 tests)	Data cleaning, features, training, tree-evaluator parity, dispatch benchmark sanity	CI on every push; weekly
mesh-android unit tests (11)	Envelope validation, framing and reassembly, 'already have' digest	APK workflow
Type-check + production build	Whole web app	CI on every push

Environment variables and workflows

Name	Needed for	Now
DATABASE_URL / DB_URL / DB_POSTGRES_URL / POSTGRES_URL	Neon database (any one; the Vercel integration sets these)	LIVE
NEXTAUTH_SECRET · NEXTAUTH_URL	Session key · public address for OAuth	LIVE
GOOGLE_CLIENT_ID · GEMINI_API_KEY	Google sign-in · Gemini briefings	LIVE
VOLENTIFY_ADMIN_EMAILS · IMPOSSIBL_API_KEY · MESH_RELEASE_REPO	Auto-verified admins · hosted triage · APK repo (default Gyanvir/Volentify)	OPTIONAL
VOLENTIFY_OFFLINE	Set to 1 to stop all outbound feed calls (tests)	OPTIONAL
Workflow: CI · Volentify Mesh APK	every push · changes under mesh-android/	LIVE
Workflow: Live feeds	every 15 min → /api/live/refresh (repo variable VOLENTIFY_URL, default volentify.vercel.app)	LIVE
Workflow: Train cyclone models · Collect research archive	weekly · every 3 hours	LIVE

14. What changed in this edition

Area	Change	State
Database	Neon only; every Supabase reference removed (code, docs, local config).	DONE
Real data	Built-in demo dataset deleted; incidents from GDACS, USGS, NASA EONET; demo rows purged from Neon by migration; official warnings enriched and expiry-filtered; facilities from OpenStreetMap; fake samples removed from tools.	DONE
Feed refresh bug	Fresh serverless instances skipped the first 15 minutes of refreshes (monotonic clock starts near zero); fixed and covered by a test.	FIXED
Tactical map	Imagery now drives the live tactical map on the Intelligence page; Satellite switch, base-map switcher, opacity and imagery date on the main map.	DONE
Mesh latency	45 s per-peer cool-down removed; event-driven relay, low-latency discovery, digest, parallel sessions, connection priority.	DONE
Mesh GPS	Location permission requested correctly on Android 12+; live location status; accuracy recorded.	DONE
Mesh crash	Writes over 512 bytes threw on Android 13/14 (present before); now capped.	FIXED
Auto-sync	Reports and messages upload whenever mobile data or Wi-Fi is available, token optional, and join the incident timeline on the website.	NEW
Analytics	Vercel Web Analytics enabled.	NEW

15. Where we are

Area	State
Website (28 pages), design system, tactical map with satellite imagery, radar and 3D terrain	DONE
API (46 endpoints) with Neon, migrations, live feeds and durability rules	DONE
Real data only: live hazard feeds, official warnings, OpenStreetMap facilities	DONE
Accounts, roles, onboarding, muster passes, admin verification	DONE
Cyclone ML models trained on real data, evaluated on held-out storms, retrained weekly	DONE
Dispatch engine with weekly benchmark against exact optimisation	DONE
Mesh Android app: fast BLE relay, messages, offline storage, automatic sync, GPS, releases	DONE

Item	What is needed	State
Mesh field test	Two or three phones on the new APK with mobile data off; confirm hop time, then turn data on and watch reports reach the incident timeline	ACTION
Old app installs	Phones on builds before the latency fix still wait 45 s per peer; update every phone	ACTION
Stable APK signing	Add the four signing secrets (mesh-android/signing/HOW-TO-USE.txt); uninstall the old build once	ACTION
Hospitals & shelters	Depends on Overpass answering Vercel; mirrors added — confirm /resources fills	PARTIAL
Android 10–11 background scanning	Needs a location foreground-service type to scan while backgrounded	PARTIAL
Storm-surge accuracy	No open surge observations for India; needs tide-gauge data (e.g. INCOIS)	PARTIAL
SitRep DistilBERT · trained surge model · iOS app · push/SMS alerts	Not built yet	NOT BUILT

For the research paper

- Cite the data: IBTrACS version and date span from the model card; GDACS, USGS and NASA EONET for incidents; NDMA SACHET for alerts; OpenStreetMap (ODbL) for facilities.
- Use only measured figures: cyclone metrics (section 8) and the dispatch benchmark (section 9). Treat PRD performance numbers as targets.
- The research archive has run since 1 November 2026; exclude snapshots taken before the 2 November demo purge.
- Check every reference before citing it. "Sperling (2026)" is used as a name in the code and PRD but has not been verified as a published source.